Lab2 进程通信与内存管理

软中断通信

编写程序：

```c
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
#include <stdlib.h>
#include <signal.h>
int flag = 0;
void inter_handler(int sig) {
    printf("\n %d stop test.\n", sig);
    flag = sig;
}
void waiting() {
    // alarm(5);
    // while (!flag) {
    //     pause();
    // }
    sleep(5);
    if (flag == 0) kill(getpid(), SIGINT);
}
int main() {
    // TODO: 五秒之后接收到两个信号
    pid_t pid1 = -1, pid2 = -1;
    while (pid1 == -1)
        pid1 = fork();
    if (pid1 > 0) {
        while (pid2 == -1)
            pid2 = fork();
        if (pid2 > 0) {
            // 父进程
            // SIGINT = 2, SIGQUIT = 3
            signal(SIGINT, inter_handler);
            signal(SIGQUIT, inter_handler);
            signal(SIGALRM, inter_handler);
            waiting();
            kill(pid1, SIGSTKFLT);
            kill(pid2, SIGCHLD);
            waitpid(pid1, NULL, 0);
            waitpid(pid2, NULL, 0);
            printf("\nParent process is killed!!\n");
        }
        else {
            // 子进程 2
            signal(SIGINT, SIG_IGN);
            signal(SIGQUIT, SIG_IGN);
            signal(SIGCHLD, inter_handler);
            pause();
            printf("\nChild process2 is killed by parent!!\n");
            return 0;
        }
```

```
    }
    else {
        // 子进程1
        signal(SIGINT, SIG_IGN);
        signal(SIGQUIT, SIG_IGN);
        signal(SIGSTKFLT, inter_handler);
        pause();
        printf("\nChild process1 is killed by parent!!\n");
        return 0;
    }
    return 0;
}

```

其中，

```
signal(SIGQUIT, SIG_IGN);
signal(SIGINT, SIG_IGN);
signal(SIGALRM, SIG_IGN);
```

用于在子进程中忽略`SIGQUIT`、`SIGINT`、`SIGALRM`信号。

（1）你最初认为运行结果会怎么样？写出你猜测的结果。

答：在 5 秒内按下 Ctrl+C（产生 SIGINT）或 Ctrl+\（产生 SIGQUIT），父进程会捕获信号（inter_handler 设置 flag），随后父进程通过 kill通知两个子进程结束；子进程收到信号后打印“Child processX is killed by parent!!”并退出；父进程在等待两个子进程结束后打印“Parent process is killed!!”并退出。如果在 5 秒内无键盘中断，waiting() 会在 sleep(5) 后使父进程自发接收 SIGINT（通过 kill(getpid(), SIGINT)），从而触发与上面相同的流程。

（2）实际的结果什么样？有什么特点？在接收不同中断前后有什么差别？请将5秒内中断和5秒后中断的运行结果截图，试对产生该现象的原因进行分析。

实际结果：若按下 Ctrl+C（SIGINT）或 Ctrl+\（SIGQUIT），父进程捕获并进入 inter_handler，然后继续执行 waiting() 后续逻辑，通过 sleep(5) 实现等待并自杀逻辑；如果 flag 已被设置，可以跳过自杀分支，父进程随后的 kill(pid1, SIGSTKFLT) / kill(pid2, SIGCHLD) 会被执行以通知子进程。如果 5 秒内没有外部中断，父进程会在 waiting()后向自身发送 SIGINT，触发 inter_handler，继而运行父进程向子进程发送信号并等待子进程结束的流程。由于子进程对 SIGINT、SIGQUIT 设为 SIG_IGN，所以若父进程直接发送 SIGINT/SIGQUIT，子进程不会响应。但父选择向子发送它们各自登记的信号（子1的16号信号SIGSTKFLT、子2的17号信号SIGCHLD），因此子能在 pause() 中被唤醒，执行其处理函数并退出。

特点与差异原因：

①信号来源不同但处理流程一致：键盘中断由终端发送给父进程，父进程捕获后触发后续；超时则是父进程自己调用 kill(getpid(), SIGINT)；两者最后都通过父发向子进程的 kill(pid, ...) 来通知子进程结束，因此表面上观察到的父、子输出顺序是一致的。

②父—子进程对某些信号的忽略：子把 SIGINT、SIGQUIT 设为 SIG_IGN，这样在父进程接收到或自发产生这些信号时，若父不再转发子这些信号，则子不会被中断。代码通过向子发送特定信号确保子能被正确唤醒并退出。

（3）改为闹钟中断后，程序运行的结果是什么样子？与之前有什么不同？

结果：若将等待机制改为使用 alarm(5)，则父进程会在 5 秒后由内核发送 SIGALRM 到父进程，而不是父进程自己 kill(getpid(), SIGINT)。父在 inter_handler 中设置 flag，接着父会发送对子进程的结束信号并等待其退出，总体流程与“自发 kill(getpid(), SIGINT)”类似。

主要差别：

①信号来源不同：SIGALRM 是由内核定时器触发，更清晰可靠；而 kill(getpid(), SIGINT) 是程序自己向自身发送 SIGINT，语义上显得不自然。

②如果使用 SIGALRM，无需对子进程进行特殊信号语义映射；可以直接让父在 SIGALRM 处理器中执行通知子进程的逻辑。SIGALRM 避免了在代码里人为制造 SIGINT 的情况，从可读性与维护性上更优。

（4）kill命令在程序中使用了几次？每次的作用是什么？执行后的现象是什么？

答：kill 被使用了三次：

①kill(getpid(), SIGINT);（位于 waiting() 中，仅在 5 秒超时且 flag == 0 时执行）

作用：程序向自身发送 SIGINT，触发父进程的 inter_handler；等价于“父进程自己产生一个中断”以进入信号处理分支。

执行后的现象：父进程进入 inter_handler（输出信号号、设置 flag），随后父进程继续执行发送给子进程的 kill 操作，唤醒/通知子进程退出。

②kill(pid1, SIGSTKFLT);（父在 waiting() 返回后对子进程 1 发）

作用：向子进程 1 发送16号信号，子进程 1 对该信号安装了 inter_handler，因此子进程 1 将从 pause() 唤醒并在处理器返回后打印“Child process1 is killed by parent!!”然后退出。

执行后的现象：子进程 1 被唤醒并退出；父通过 waitpid(pid1, …) 等待其结束。

③kill(pid2, SIGCHLD);（父对子进程 2 发）

作用：向子进程 2 发送17号信号，子进程 2 对该信号安装了 inter_handler，因此子进程 2 会被唤醒并退出。

执行后的现象：子进程 2 打印“Child process2 is killed by parent!!”并退出；父通过 waitpid(pid2, …) 等待其结束。

（5）使用kill命令可以在进程的外部杀死进程。进程怎样能主动退出？这两种退出方式哪种更好一些?

进程主动退出方法：调用 exit()、自然执行到末尾或通过父进程通知后由子自行 exit()。

- 优点：可以做清理工作（释放资源、写日志、关闭文件描述符、通知其他进程等），允许程序以受控方式终止，避免资源泄露或部分写入。
- 缺点：若进程遇到不可恢复错误或被恶意阻止主动退出，可能无法按期终止。

外部 kill：

- 优点：能强制终止无法响应的进程（例如死锁或被阻塞的场景），管理员可以通过 SIGKILL 立即终止问题进程。
- 缺点：若使用强制信号，进程无法做善后处理；若使用非强制信号，需要进程正确处理该信号才能优雅退出；另外，误用或语义不当的信号会造成混淆。

结论：进程主动退出更好，因为其更安全、更可控。当进程不能主动终止或出现异常时，才使用外部 kill 强制结束。

管道通信

根据所给代码补充完在父(子)进程中读写的部分代码

```
#include <unistd.h>
#include <signal.h>
#include <stdio.h>
#include <stdlib.h>
#include <sys/wait.h>
#include <string.h>
int pid1, pid2; // 定义两个进程变量
int main() {
    int fd[2];
    char InPipe[4001]; // 定义读缓冲区
    char OutPipe[4001]; // 定义写缓冲区
    char c1 = '1', c2 = '2';
    if (pipe(fd) == -1) { // 创建管道
        perror("pipe");
        exit(1);
    }
    while ((pid1 = fork()) == -1); // 如果进程1创建不成功,则空循环
    // 如果子进程1创建成功,pid1为进程号
    if (pid1 == 0) {
        lockf(fd[1], 1, 0);          // TODO:锁定管道
        // 将"Child process 1 is sending message!"写入OutPipe
        snprintf(OutPipe, sizeof(OutPipe), "Child process 1 is sending message!");
        write(fd[1], OutPipe, strlen(OutPipe));

        // TODO:分2000次每次向管道写入字符'1'
        // for (int i = 0; i < 2000; i++) {
        //     write(fd[1], &c1, 1);
        // }

        sleep(5); // 等待读进程读出数据
        lockf(fd[1], 0, 0);          // TODO:解除管道的锁定
        exit(0);  // 结束进程1
    } else {
        while ((pid2 = fork()) == -1); // 若进程2创建不成功,则空循环
        if (pid2 == 0) {
            lockf(fd[1], 1, 0);
            // 将"Child process 2 is sending message!"写入OutPipe
            snprintf(OutPipe, sizeof(OutPipe), "Child process 2 is sending message!");
            write(fd[1], OutPipe, strlen(OutPipe));

            // TODO:分2000次每次向管道写入字符'2'
            // for (int i = 0; i < 2000; i++) {
            //     write(fd[1], &c2, 1);
            // }
            sleep(5);
```

```
            lockf(fd[1], 0, 0);
            exit(0);
        } else {
            wait(0);                // TODO:等待子进程1结束
            wait(0);                // 等待子进程2结束
            read(fd[0], InPipe, 4000);      // TODO:从管道中读出4000个字符
            InPipe[4000] = '\0';            //  TODO:加字符串结束符
            printf("%s\n", InPipe); // 显示读出的数据
            exit(0);                // 父进程结束
        }
    }
}

```

在子进程中，删除lockf函数

```
//lockf(fd[1], 1, 0);
、、、
//lockf(fd[1], 0, 0);
```

则管道读写的互斥和同步出现问题，加上之后输出结果正确。

(1)你最初认为运行结果会怎么样?

答：如果代码在写操作上使用了锁，预期父进程读取到的 InPipe 将包含两个子进程的写入内容，且两段写入不会相互交错。如果取消锁并且两个子进程同时频繁写入，写操作可能会因为多个写者同时向同一管道写入导致交错或出现错位，造成读取到的内容不完整或顺序不确定。父进程在 wait() 等待子进程结束后再读，避免了读取时写端仍打开导致读取阻塞，但若父先读而写端仍打开，read() 可能阻塞直到有足够数据或写端关闭。

(2)实际的结果什么样？有什么特点？试对产生该现象的原因进行分析。

实际结果：

①有锁情形：写入被序列化，父进程读取到的数据为“子进程 1 的消息 + 子进程 2 的消息”的拼接。当子写入短字符串"Child process X is sending message!"时，数据整段写入，不会被其他进程中断，因此父读到的字符串整洁且完整。

②无锁情形：若两个子进程都进行大量写入，写操作之间会交错或部分混合。因此不加锁会出现竞争，读取到的内容顺序不确定、可能被打断或混合，从而产生数据错乱或并发写入的污染。

③读阻塞与进程退出：父进程先 wait() 等待两个子退出，子退出后写端文件描述符全部关闭，read(fd[0], InPipe, 4000) 会读出所有管道中数据并返回，随后父打印并退出。这种先等待再读避免了读取阻塞的问题，因为写端已关闭，read 能立即返回已有数据。

(3)实验中管道通信是怎样实现同步与互斥的？如果不控制同步与互斥会发生什么后果?

实现方式：

①文件锁（lockf）：在写端上加锁以保证一次只有一个进程进行写入，从而实现互斥。适用于写入较大数据并需保证写入原子性时的序列化。

②通过父进程中使用的 wait(0) 系统调用，让父进程在执行read之前阻塞等待两个子进程的终止，确保数据在读取之前已被完整写入管道。

若不控制同步互斥会发生：

①数据交错/损坏：多个写者同时写入大于PIPE_BUF的数据会发生拆分与交错，导致读取端接收到混合数据。

②顺序不可预测：读取端无法保证接收到的消息顺序，因为不同子进程写入顺序不确定。

页面置换

```c
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#define MAX_PAGES 100    // 页面序列最大长度
#define MAX_FRAMES 10    // 物理块最大数量
int page_seq[MAX_PAGES]; // 页面访问序列
int frame[MAX_FRAMES];   // 当前内存块
int page_num;            // 页面序列长度
int frame_num;           // 内存块数量

// 打印当前内存状态
void print_frames() {
    for (int i = 0; i < frame_num; i++) {
        if (frame[i] == -1) printf(" - ");
        else printf(" %d ", frame[i]);
    }
    printf("\n");
}

// FIFO 页面置换算法
void fifo() {
    int page_faults = 0;
    int next_to_replace = 0; // FIFO替换指针

    for (int i = 0; i < frame_num; i++)
        frame[i] = -1;

    printf("\n*** FIFO 页面置换过程 ***\n");
    for (int i = 0; i < page_num; i++) {
        int page = page_seq[i];
        int found = 0;
        for (int j = 0; j < frame_num; j++) {
            if (frame[j] == page) {
                found = 1;
                break;
            }
        }

        if (!found) {
            frame[next_to_replace] = page;
            next_to_replace = (next_to_replace + 1) % frame_num;
            page_faults++;
            printf("缺页 -> 调入页面 %d :", page);
        } else {
            printf("命中 -> 页面 %d :", page);
```

```
        }
        print_frames();
    }

    printf("\nFIFO缺页次数: %d, 缺页率: %.2f%%\n",
           page_faults, 100.0 * page_faults / page_num);
}

// LRU 页面置换算法
void lru() {
    int page_faults = 0;
    int last_used[MAX_FRAMES]; // 每个块最近使用的时间

    for (int i = 0; i < frame_num; i++) {
        frame[i] = -1;
        last_used[i] = -1;
    }

    printf("\n*** LRU 页面置换过程 ***\n");
    for (int t = 0; t < page_num; t++) {
        int page = page_seq[t];
        int found = 0;
        for (int j = 0; j < frame_num; j++) {
            if (frame[j] == page) {
                found = 1;
                last_used[j] = t; // 更新最近使用时间
                break;
            }
        }

        if (!found) {
            int replace_index = 0, oldest = last_used[0];
            for (int j = 1; j < frame_num; j++) {
                if (frame[j] == -1) { replace_index = j; break; }
                if (last_used[j] < oldest) {
                    oldest = last_used[j];
                    replace_index = j;
                }
            }
            frame[replace_index] = page;
            last_used[replace_index] = t;
            page_faults++;
            printf("缺页 -> 调入页面 %d :", page);
        } else {
            printf("命中 -> 页面 %d :", page);
        }
        print_frames();
    }

    printf("\nLRU缺页次数: %d, 缺页率: %.2f%%\n",
           page_faults, 100.0 * page_faults / page_num);
}

int main() {
    int choice, mode;

```

```
    printf("请输入页面引用序列长度: ");
    scanf("%d", &page_num);
    printf("请输入物理块数量: ");
    scanf("%d", &frame_num);

    printf("选择页面序列生成方式: 1.手动输入  2.随机生成\n");
    scanf("%d", &mode);

    if (mode == 1) {
        printf("请输入页面引用序列（以空格分隔）:\n");
        for (int i = 0; i < page_num; i++)
            scanf("%d", &page_seq[i]);
    } else {
        srand(time(NULL));
        for (int i = 0; i < page_num; i++)
            page_seq[i] = rand() % 10; // 0~9的随机页号
        printf("随机生成的页面序列为:\n");
        for (int i = 0; i < page_num; i++)
            printf("%d ", page_seq[i]);
        printf("\n");
    }

    printf("\n选择算法: 1.FIFO  2.LRU\n");
    scanf("%d", &choice);

    if (choice == 1) fifo();
    else if (choice == 2) lru();
    else printf("无效选择。\n");

    return 0;
}

```

结果:

1.FIFO: 当需要调入页面到内存时，选择最先调入页面的那个置换掉。

2.LRU: 当需要调入页面到内存时，选择最近最少使用的页面置换掉。

3.当内存页面数增加了以后，命中率反而降低了。

4.具有局部性良好的数据，LRU命中率高于FIFO算法

结果解释:

1.使用FIFO算法，创建一个队列来管理所有的内存页面，置换的是队列的首个页面。当需要调入页面到内存时，就将它加到队列的尾部。

2.LRU 置换将每个页面与它的上次使用的时间关联起来。当需要置换页面时，LRU 选择最长时间没有使用的页面。代码中为每个页表条目关联一个使用时间域clock，并为 CPU 添加一个全局的计数器，每次内存引用都会递增时钟，并将时钟寄存器的内容复制到对应页表条目的使用时间域，这样每次最小的时钟域就是要被置换的页面。

3.Belady 异常指的是， 对于FIFO算法，随着内存分配帧数量的增加，命中率可能会下降。

4.LRU具有良好的局部性，对于最近出现过的页面，更不可能被换出，而局部性数据一般会多次重复访问，所以命中率会更高。